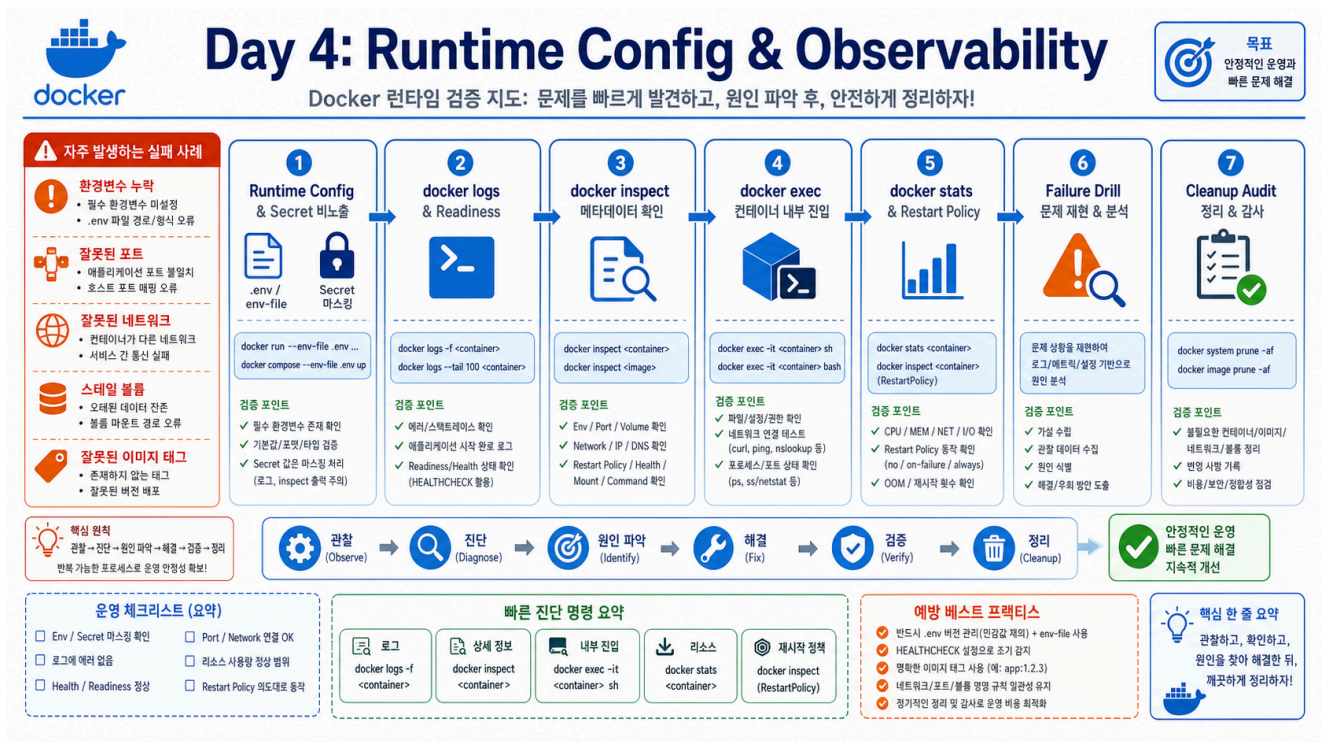


1교시|: Runtime config, env file, secret masking



수업 목표

- runtime config가 image build와 다른 단계임을 설명한다.
- e와 --env-file로 env를 주입하고 출력으로 확인한다.
- env 값이 image 안에 굳어진 값인지, 실행 시점에 들어간 값인지 구분한다.
- .env.example과 실제 .env의 역할을 구분한다.
- secret 값을 문서와 screenshot에 남기지 않는 기준을 적용한다.

개념 설명

Day 3에서 image를 만들었다면 Day 4는 그 image를 어떤 조건으로 실행할지 다룬다. 같은 image라도 APP_ENV=dev로 실행할 때와 APP_ENV=prod로 실행할 때 의미가 달라진다. 이 차이를 image를 다시 build해서 만들면 환경마다 image가 늘어나고, secret이 image layer에 남을 위험도 커진다.

Runtime config는 container를 시작할 때 들어가는 실행 조건이다. 대표적으로 env, port, volume, network, restart policy가 있다. 이 교시는 env부터 시작한다. 핵심은 값이 들어갔다가 아니라 어떤 방식으로 들어갔고, 어디에 남았고, 어떻게 확인했는가다.

현업 시나리오로 보면 더 분명하다. 같은 웹 애플리케이션 image를 개발 환경에서는 APP_ENV=dev, staging에서는 APP_ENV=staging, 운영에서는 APP_ENV=prod로 실행할 수 있다. 이때 환경마다 image를 다시 만들면 dev image, staging image, prod image가 서로 달라지고, 어떤 image가 실제

코드 변경 때문인지 설정 차이 때문인지 추적하기 어려워진다. 그래서 image는 가능한 한 동일하게 두고, 실행 조건을 runtime config로 바꾸는 방향을 기본값으로 둔다.

학생이 이 교시에서 잡아야 할 기준은 다음 한 문장이다.

image는 실행할 프로그램을 담고, runtime config는 그 프로그램이 어떤 환경으로 실행될지 정한다.

이 기준이 잡히면 Day 4의 나머지 내용도 자연스럽게 이어진다. logs는 실행 결과를 보는 도구이고, inspect는 어떤 runtime config가 적용됐는지 보는 도구이고, exec는 container 안에서 실제 상태를 확인하는 도구다.

환경설정을 파일로 로드할 때는 `docker run --env-file ./path/to/.env ...`를 쓴다. Docker는 이 파일을 container 생성 시점에 읽어서 container 환경변수로 넣는다. 파일을 나중에 수정해도 이미 만들어진 container에는 자동 반영되지 않는다. 값을 바꿨다면 container를 다시 생성해야 한다.

env file의 기본 형식은 다음과 같다.

형식	의미
APP_ENV=practice	값을 직접 지정
FEATURE_FLAG=on	값을 직접 지정
LOCAL_ONLY_KEY	host shell에 export된 값을 가져옴
# comment	comment로 무시

-e와 --env-file의 차이는 사용 목적이다.

방식	예시	적합한 상황	주의할 점
-e KEY=value	-e APP_ENV=practice	한두 개 값을 빠르게 바꿔 실행	명령줄과 shell history에 값이 남기 쉬움
--env-file FILE	--env-file .env	여러 설정을 파일로 묶어 재사용	파일을 명시하지 않으면 docker run이 자동으로 읽지 않음
-e KEY	-e APP_ENV	host shell의 export 값을 container로 전달	host에 값이 없으면 container에도 기대한 값이 없음

.env는 다음처럼 한 줄에 하나씩 기록한다.

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
# DEBUG=true
```

이 파일을 --env-file로 넘기면 container 안에서는 환경변수로 보인다. 파일 자체가 container 안에 복사되는 것이 아니라, 파일의 key/value가 container 환경으로 주입된다.

환경별로 파일을 나누는 것도 가능하다. 예를 들어 같은 image를 개발, staging, 운영 조건으로 나누고 싶다면 다음처럼 파일을 둘 수 있다.

```
.env.dev
.env.staging
.env.prod
```

각 파일은 같은 key를 가지되 값만 다르게 둔다.

```
# .env.dev
APP_ENV=dev
FEATURE_FLAG=on
API_BASE_URL=http://localhost:3000
```

```
# .env.staging
APP_ENV=staging
FEATURE_FLAG=on
API_BASE_URL=https://staging.example.com
```

```
# .env.prod
APP_ENV=prod
FEATURE_FLAG=off
API_BASE_URL=https://api.example.com
```

실행할 때는 어떤 환경 파일을 쓸지 명시한다.

```
docker run --rm --env-file .env.dev alpine:3.20 env
docker run --rm --env-file .env.staging alpine:3.20 env
docker run --rm --env-file .env.prod alpine:3.20 env
```

이 패턴은 Docker에서 끝나지 않는다. Kubernetes에서는 ConfigMap/Secret으로 환경별 설정을 분리하고, Terraform에서는 dev.tfvars, staging.tfvars, prod.tfvars처럼 환경별 variable 파일로 같은 생각을 확장한다. 지금 배우는 핵심은 특정 파일 이름이 아니라 ****코드와 image는 최대한 동일하게 두고, 환경 차이는 설정으로 분리한다****는 원칙이다.

실습 명령

```
cd /mnt/d/paperclip
docker run --rm -e APP_ENV=practice -e FEATURE_FLAG=on alpine:3.20 env | sort
| grep -E 'APP_ENV|FEATURE_FLAG'
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
```

같은 image를 다른 env로 실행해 차이를 확인한다.

```
docker run --rm -e APP_ENV=dev alpine:3.20 sh -c 'echo "image=alpine:3.20
APP_ENV=$APP_ENV"'
docker run --rm -e APP_ENV=prod alpine:3.20 sh -c 'echo "image=alpine:3.20
APP_ENV=$APP_ENV"'
```

Expected:

```
image=alpine:3.20 APP_ENV=dev
image=alpine:3.20 APP_ENV=prod
```

해석: image는 둘 다 alpine:3.20으로 같지만 runtime config만 다르다. 이 차이를 이해해야 이후 nginx, PostgreSQL, Compose, Kubernetes에서도 config와 artifact를 섞지 않는다.

env file 확인

```
sed -n '1,120p' week2/day4/labs/env-report/.env.example
cp week2/day4/labs/env-report/.env.example week2/day4/labs/env-report/.env
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 env |
sort | grep -E 'APP_ENV|FEATURE_FLAG'
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
```

.env 값이 container 안에서 어떻게 보이는지 확인하되, 기록용 출력은 masking한다.

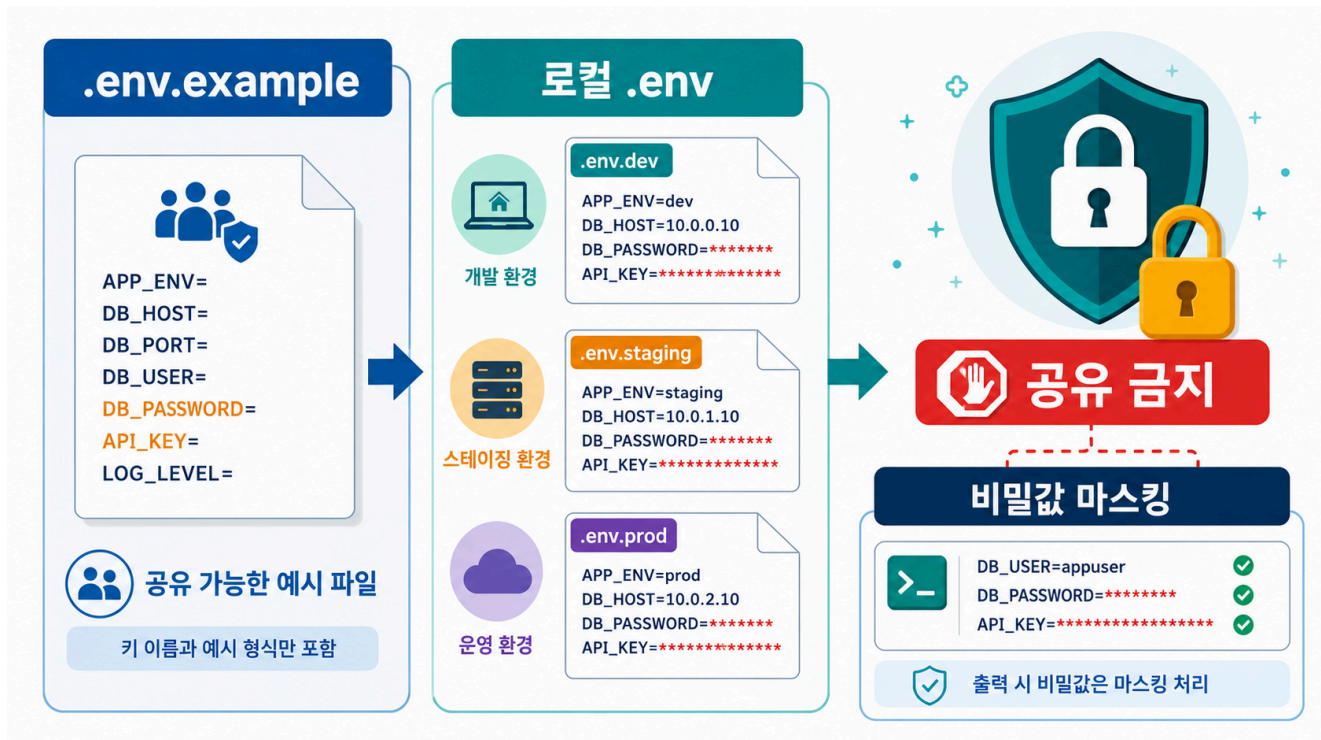
```
docker run --rm --env-file week2/day4/labs/env-report/.env alpine:3.20 sh -c
'echo "APP_ENV=$APP_ENV FEATURE_FLAG=$FEATURE_FLAG DB_PASSWORD=$DB_PASSWORD" '
\
| sed -E 's/DB_PASSWORD=.* /DB_PASSWORD=***masked***/'
```

Expected:

```
APP_ENV=practice FEATURE_FLAG=on DB_PASSWORD=***masked***
```

실제 값이 container 안에 주입되는 것은 맞지만, 문서나 screenshot에는 password 값을 남기지 않는다.

.env.example과 secret masking



.env.example은 팀원에게 필요한 key 이름과 형식을 알려주는 파일이다. 실제 password, token, cloud key를 담는 파일이 아니다. 반대로 .env는 로컬 실행을 위한 실제 값이 들어갈 수 있으므로 공개 repository, README, screenshot, 질문 게시글에 그대로 올리면 안 된다.

초보자는 실습용이니까 괜찮다고 생각하기 쉽다. 하지만 습관이 중요하다. 수업용 password라도 그대로 캡처해 공유하는 방식에 익숙해지면, 이후 Docker Hub token, AWS access key, database password도 같은 방식으로 노출될 수 있다.

실제 사고 연결: TVING 자격증명 노출 논란

2026년 6월 TVING 개인정보 유출 사고 보도에서는 단순 DB 유출을 넘어 GitHub 자격증명 노출, AWS access key 폐기, 클라우드 접근 권한 관리 부실이 쟁점으로 제기됐다. 일부 보도는 소스코드에 평문 자격증명을 적어두는 하드코딩 관행을 주요 원인 후보로 설명했다. 다만 공개 기사만으로는 실제 침입 경로와 최종 원인이 확정됐다고 단정할 수 없으므로, 수업에서는 "소스 저장소와 runtime secret 관리가 사고 대응의 핵심 쟁점이 된 사례"로 다룬다.

이 사례가 Day 4의 .env 실습과 완전히 같은 유형은 아니다. TVING 보도에서 언급된 것은 GitHub, AWS access key, 내부 시스템 접근 권한 같은 더 큰 범위의 credential 관리 문제다. 하지만 원칙은 같

다. secret이 source code, image layer, repository, terminal screenshot, log, issue에 남으면 공격자는 복잡한 취약점을 찾지 않고도 내부 시스템으로 들어갈 수 있다.

사고 쟁점	Day 4에서 연결할 원칙
GitHub 자격증명 노출 가능성	repository에는 실제 .env, access key, token을 올리지 않는다.
AWS access key 폐기/교체	노출이 의심되면 값을 숨기는 것이 아니라 즉시 revoke/rotate한다.
소스코드 하드코딩 우려	credential은 코드 상수가 아니라 env, secret manager, CI/CD secret으로 주입한다.
내부 시스템 직접 접근 정황	credential 하나가 DB, storage, admin API 등 넓은 권한으로 이어지지 않게 least privilege를 적용한다.
사고 후 원인 추적 어려움	누가 어떤 secret을 어디서 사용했는지 audit log와 runbook으로 남긴다.

수업에서는 AWS key를 만들지 않는다. 대신 DB_PASSWORD 예시를 통해 같은 습관을 훈련한다. 실습 값이라도 공개 출력에 그대로 남기지 않고, 실제 운영 secret이라면 애초에 repository와 image에 들어가지 않게 설계한다.

공유할 수 있는 파일은 보통 값이 비어 있거나 placeholder만 있는 예시 파일이다.

```
# .env.prod.example
APP_ENV=prod
FEATURE_FLAG=off
API_BASE_URL=https://api.example.com
DB_PASSWORD=replace-me-securely
```

이 파일은 prod에는 어떤 key가 필요한가를 알려주는 문서 역할을 한다. 실제 .env.prod는 로컬 또는 배포 시스템 안에서만 관리한다.

Masking script로 공유 가능한 출력만 남긴다.

```
chmod +x week2/day4/labs/env-report/report.sh
docker run --rm --env-file week2/day4/labs/env-report/.env \
-v "$PWD/week2/day4/labs/env-report:/work:ro" \
alpine:3.20 /work/report.sh
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
```

문서화 기준:

문서에 남겨도 되는 것	문서에 남기면 안 되는 것
DB_PASSWORD라는 key 이름	실제 password 값
--env-file .env 사용 방식	.env 전체 내용
DB_PASSWORD=***masked***	terminal에 찍힌 실제 token
.env.example	개인 .env
.env.prod.example	실제 .env.prod

환경별 secret 판단:

파일	공유 여부	이유
.env.example	가능	key 이름과 placeholder만 포함
.env.dev	보통 로컬 전용	개인 개발값이 들어갈 수 있음
.env.staging	제한 공유	staging credential 포함 가능
.env.prod	공유 금지	운영 secret 포함 가능

운영 시나리오

같은 image를 dev, staging, prod에서 모두 쓴다고 가정한다. 학생은 다음 질문에 답해야 한다.

질문	판단
image를 새로 build해야 하는가	아니다. 실행 조건만 바꾼다.
env file을 바꿨는데 기존 container 값이 바뀌는가	아니다. container를 다시 만들어야 한다.
prod password를 .env.prod.example에 넣어도 되는가	아니다. key 이름과 placeholder만 둔다.
runtime config가 잘못됐을 때 첫 증거는 무엇인가	docker inspect, docker logs, masking된 env 확인이다.

판단 기준

질문	확인 방법	좋은 답
env가 image build 없이 바뀌는가	같은 image를 다른 -e로 실행	실행 시점 값이 달라짐
-e와 --env-file 중 무엇을 쓰는가	값 개수와 재사용 여부 확인	임시 1개는 -e, 여러 설정은 env file
.env는 어떻게 쓰이는가	--env-file .env로 container 실행	파일 내용이 env로 주입됨
환경별 설정을 분리할 수 있는가	.env.dev, .env.staging, .env.prod 비교	같은 key, 다른 value로 환경 차이를 표현
env 값이 어디에 남는가	shell history, README, screenshot 확인	실제 secret 값은 남기지 않음
env file은 공유 가능한가	.env.example과 .env 구분	example만 공유, 실제 값은 local
env file 수정이 언제 반영되는가	container 재생성 후 확인	기존 container에는 자동 반영되지 않음
image와 runtime config를 구분하는가	같은 image를 다른 env로 실행	image는 같고 실행 조건만 달라짐
credential 사고가 나면 무엇을 하는가	key 위치, 노출 범위, 사용 로그 확인	masking이 아니라 revoke/rotate와 권한 축소가 필요함

참고 사례 링크

- 보안뉴스, "티빙 사태, DB 유출보다 깃허브에 주목할 이유":
<https://m.boannews.com/html/detail.html?idx=144054>
- 이데일리, "티빙 해킹, 단순 정보유출 넘어 클라우드 계정 관리 논란으로 확산":
<https://v.daum.net/v/Y2xTjfMVDp>
- AWS 기술 블로그, "인터넷에 노출된 자격증명 탐지하기":
<https://aws.amazon.com/ko/blogs/tech/detect-exposed-security-credential/>

다음 연결

다음 교시는 container를 실행하고 logs와 HTTP 응답을 분리해 정상 여부를 판단한다.